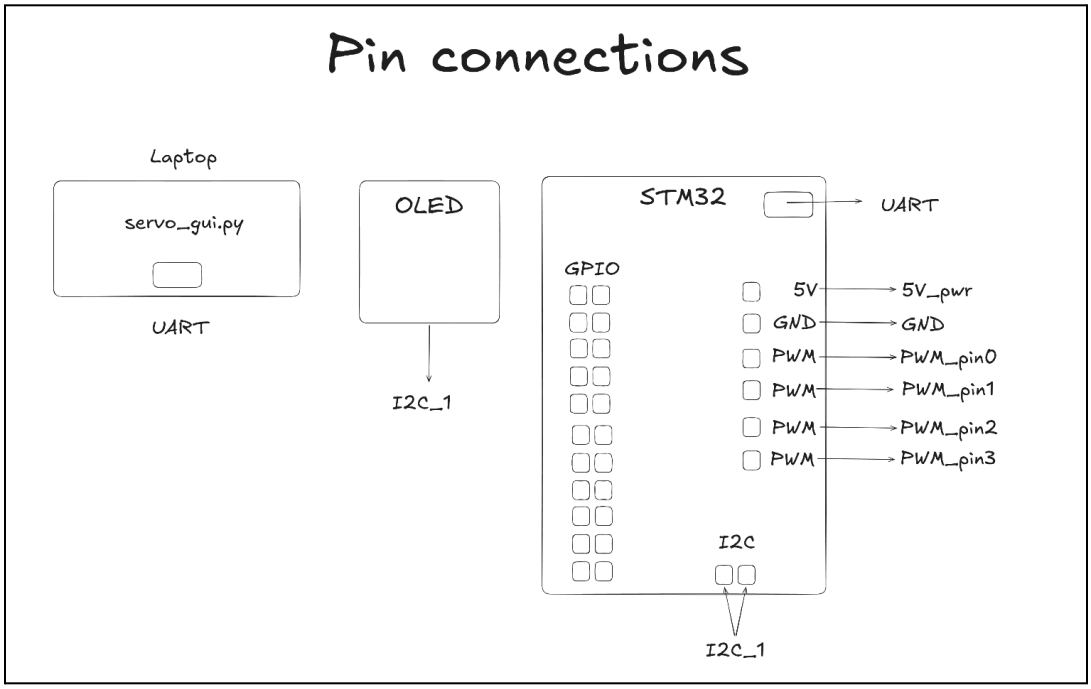
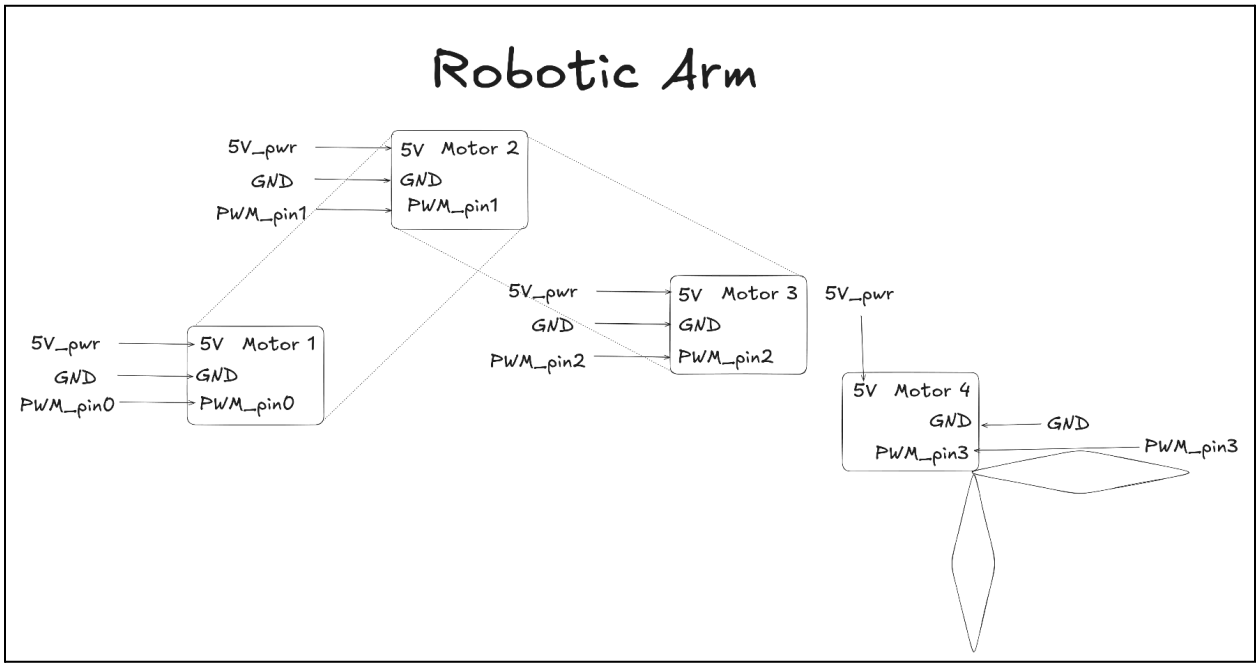
Lancelot Shih 1008759817 Kenny Guo 1010207312

Section 1 — Project Overview & Analysis





We created a 3-degree-of-freedom (3DOF) robotic arm using a 3D printed mechanical frame, actuated by 4 MG90S micro-servos and controlled by an STM32F446ZE Nucleo-144 microcontroller. The system supports two modes of control: direct joint-angle control, and cartesian coordinate based control via an inverse kinematics solver running on a host PC. A host side python application accepts user input, computes target joint angles when operating in IK mode, encodes them as an ASCII command string and transmits them to the STM32 over a UART link at 115,200 baud via a USB to serial bridge. On the STM32, a main loop state

machine parses incoming commands and updates PWM duty cycles to adjust the servo angles. Lastly, the SSD1306 OLED display was connected over I2C to the STM32, and would read PWM values of all four servos to provide real time feedback as a system ground truth.

The IK solver uses a geometric closed form solution for a 2 link design (shoulder + elbow) with a fixed base rotation joint. It has a parallelogram 4 bar linkage that makes the forearm servo (S3) an absolute angle controller, meaning mechanically the forearm is independently controlled by S3 and the elbow servo (S2) does not influence it through the elbow joint. This helped in our kinematic calculations to be more accurate and reduced our theoretical position error to be around 0.38cm across various poses.

Section 2 — Bandwidth / Throughput Analysis

Because our system operates on links between the PC to the STM32, the STM32 to the servos, and the STM32 to the OLED, we have 3 main connections we need to consider when doing bandwidth and throughput analysis.

UART Link

The theoretical maximum throughput at 115,200 baud rate with a standard 8N1 framing can be calculated to be around 10 bits per byte giving us 115,200/10 = 11520 bytes/s. Since a typical command format is <S#:###\n> (ex. S3:150 for elbow servo to 150 degrees), transmitting 7 bytes at the baud rate gives us 1645 packets/s as a theoretical maximum. With IK commands sending 3 inputs consecutively to adjust all servos at once, functionally that means 549 command sets per second

In practice, because the servos end up bottlenecking the UART chain, sending commands at the theoretical maximum would be unreliable. In addition, because the GUI issues a new command only on user interaction, we can only get a maximum of around 5 commands per second; far from the saturation limit of the link.

Because our user will never end up sending commands at such a fast rate, our theoretical limit is strictly just a metric to maintain the responsive feel of our system, as every command is immediately fed back with whether or not the robot succeeded in carrying out the command.

PWM Servo Link

The STM32 TIM3 timer we used is configured with a prescalar of 15 and an ARR of 19,999. Clocked from the 16 MHz HSI oscillator, our timer tick rate thus is 1MHz, or 1 tick per microsecond. So therefore our period becomes 20 ms giving our PWM a frequency of exactly 50 Hz. However, because we implemented a software slew rate limiter to prevent the servos from jerking the entire robot too quickly, our servos advance each servo by at most 1 degree per function call. So our angular velocity resultantly ends up capped at 1 degree / 20 ms period = 50 degrees/s. For instance a rotation of 90 degrees takes then 90 degrees * 20 ms/deg = 1.8 seconds at this rate.

Since this is self imposed, our limits here are technically adjustable and a theoretical maximum performance would be to simply remove the slew rate and at 600 degrees/s being the max of the servos, a maximum of 0.6 seconds of delay would be needed before the next motion

could occur. However because the slew rate must be implemented in practice, 360 degrees * 20ms/deg = 7.2 seconds of delay at max would be needed.

While we would like to operate at the theoretical limit, mechanical system wear becomes a much larger concern. With the arms being built out of PLA plastic and servo torque maximum limits being stretched at the length of our moment arms, we were forced to move the servos at a more safe velocity.

OLED Display Link

Since this display simply updates its display whenever the DMA interrupt updates with a new set of PWM values, it will operate at full I2C bus speeds and frame buffer transfer sizes.

I2C on the SSD1306 OLED runs I2C fast mode at 400 kHz, and for the 128x64 display, 1024 bytes plus the I2C frame overhead will need to be transferred. Given 8 data bits, 1 ack bit and address/command overhead, a full screen refresh takes approximately (1024 * 9) / 400 000 = 23ms, or a theoretical frame rate of 43 Hz.

Now as the servo arm moves, the OLED actually does update itself, so the angle moves like a dial. However, because the firmware also has a DMA based I2C implementation, the CPU is released immediately after initiating the transfer and continues main loop execution while the DMA controller moves the frame over the bus. So while the bus remains occupied for 23ms per refresh, it doesn't actually stall the CPU.

In practice, this means due to the DMA, the OLED imposes near-zero CPU overhead, and the constraint is mostly on the I2C bus occupancy of 23 ms per full frame refresh, but since it sits at the end of a component chain itself, does not pose any major bottlenecks.

Main Bottlenecks

The primary bottleneck in the system is the HAL polling UART receive model. Because HAL_UART_Receive blocks until each byte arrives, the CPU ends up being stalled for the duration of every command reception, preventing the slew from updating or servicing other peripherals in that time. The best single change that would thus improve performance would be to replace polling UART reception with DMA or interrupt driven reception. This would decouple communication from the control loop and allow other tasks the CPU needs to handle to occur.

Section 3 — Code Contribution Table

Module	Source type	Notes
UART Command parser (STM32)	Scratch	HAL RX polling receive, parses ASCII commands of the form "S#:###", then passes it on to the servo update routine
STM32 main state machine	Scratch	Initializes peripherals, runs main receive → parse update loop
PWM servo drivers (STM32)	Scratch	Developed firmware layer for PWM, and utilized GPIO pins for driving servo motors

OLED Display Drivers (SSD1306)	Derived from lab 4	Displaying the angle/status of all 4 servo motors in use in the arm, leveraging DMA to reduce CPU usage on handling OLED updates
Inverse kinematics solver (Python)	Scratch	Calibrated and tested from particular joints and arm design of our robot using link lengths of L1 = L2 = 8 cm
Python GUI	Scratch	Provides sliders for direct angle control and XYZ coordinate input for IK control. Formats and sends UART commands in the same parsing format accepted by the STM32 in the first module

Section 4 — Project Reflection

Originally, the goal was just to get a basic working 3DOF robotic arm with some inverse kinematic algorithm on it to gain much more fine grain dynamic control of the arm's dexterity. If we still had more time, we aimed to add a camera for computer vision and integrate that with the IK algorithm to have the arm be autonomous.

While we achieved the goal of creating the basic working 3DOF arm and also an implementation of the inverse kinematics algorithm. However, we did not get to the point of being able to implement computer vision integration.

One technical decision in terms of the design of the arm would be to adjust the inverse kinematics to be run directly on the STM32 as opposed to on the host PC before sending angles directly. This allows us to further upgrade the STM32 to have more knowledge of the arm's cartesian position, so boundary checks can be enforced in firmware outside of just angles. On-chip IK can also eliminate UART round trip time as a latency source for coordinate control.

One of the issues relevant to our project was the safety of operation in case of failure. Since the STM32 accepts and executes any well formed UART command without authentication, rate limiting, or servo orientation/bound checking, any malformed or malicious command could drive the servos past the mechanical stops of the frame and break it. To fix this we would add software enforced servo angle safety limits in the STM32 firmware and also figure out how to get the servos to operate more consistently without bugging out and performing unpredictably.

One more security issue relevant to our project was not having a kill switch readily implemented. We noticed that occasionally the only way to actually reset the system was to both reset the STM32 and also re-power the entire servo chain. If we have a physical kill switch that would fully reset the system, we could prevent needing to scramble to pull wires if the servos end up malfunctioning.

Section 5 — Workload Split

Lancelot was responsible for the majority of the integration and worked on the UART communication chain, optimization of the ground truth readings displayed on the OLED via interrupts, the inverse kinematics algorithm implementation, and built/limit tested the physical robot. Kenny was responsible for writing the firmware layer to operate the servos of the arm, and integration of the OLED, to display status of the servos during manual operation.